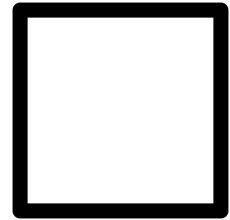


Programação e Desenvolvimento de Software 1 — Prova 2

Nome: _____

Matrícula UFMG: _____

Assinatura: _____



Instruções:

- Esta avaliação tem o valor de 30 (trinta) pontos cobrindo os conceitos de tipos primitivos, operadores, ponteiros, condicionais, repetições e variáveis indexadas.
- Todas as questões devem ser resolvidas utilizando a linguagem C.
- Se necessário, utilize a folha de papel almaço como rascunho para construir a solução. Transcreva a solução final para a prova no espaço reservado para cada questão.
- Termo de consentimento e ética: ao assinar esta prova declaro que usei somente o material permitido e que estou ciente de que *qualquer identificação de cola ou uso de celular durante a avaliação* determinará uma nota zero e encaminhamento de meu nome para processo administrativo discente junto ao Colegiado de meu curso.

Parte 1 – Batalha Naval (20 pontos)

Nas questões 1 a 4 da parte 1, iremos fazer uma versão simplificada do jogo de Batalha Naval. Os jogadores posicionam navios em um tabuleiro quadriculado de 12x12 casas representando o oceano. Nossa versão do jogo tem três tipos de navios:

- **corveta**: Ocupa uma casa no tabuleiro
- **fragata**: Ocupa duas casas no tabuleiro
- **porta-avião**: Ocupa três casas no tabuleiro

Antes do início do jogo, o computador posiciona os navios no tabuleiro. Os navios nunca ultrapassam as bordas do tabuleiro e nunca estão sobrepostos. Em nossa versão do jogo, **os navios ficam sempre na horizontal**. Por exemplo, um porta-aviões pode estar nas posições (2, 3), (2, 4), e (2, 5); enquanto uma fragata pode estar nas posições (8, 1) e (8, 2).

O jogo opera em rodadas. Em cada rodada, o jogador escolhe uma posição do tabuleiro onde deseja enviar um míssil e o jogo informa se ele acertou um navio ou não. Para afundar um navio, o jogador precisa acertar com um míssil todas as coordenadas ocupadas por um navio. Por exemplo, para afundar um porta-aviões é necessário acertar suas três posições. O jogo termina quando o jogador afunda todos os navios.

No tabuleiro, utilizaremos o caractere de ponto final ' . ' para identificar que aquela posição está vazia (não possui um navio), o caractere ' N ' para indicar que há um navio naquela posição, e o caractere ' X ' para indicar um navio atingido por um míssil naquela posição.

Questão 1 – Posicionar Navios (8 pontos)

Implemente uma função em C chamada `posicionar_navios` que recebe como parâmetros:

- Uma matriz `tabuleiro` de 12x12 do tipo `char`, representando o tabuleiro do jogo.
- Um array de strings chamado `navios`, onde cada string representa um navio e seu posicionamento. O formato de cada string é “<tipoDoNavio> <posicaoX> <posicaoY>”, onde o tipo do navio é “corveta”, “fragata” ou “porta-aviao”; e as posições são dois números entre zero e 11 indicando a coordenada mais à esquerda do navio no tabuleiro (x indica a linha e y indica a coluna, ambos começando em zero). Por exemplo: “`fragata 2 3`”.
- Um inteiro `n` que indica a quantidade de elementos no arranjo `navios` que precisam ser posicionados no tabuleiro.

Sua função deve marcar as posições dos navios no tabuleiro com o caractere 'N' e as demais posições com o caractere de ponto final.

Dicas e considerações:

1. Utilize a função `sscanf(const char *string, const char *formato, ...)`, que funciona de forma similar à função `scanf(const char *format, ...)`, mas que processa o string recebido no primeiro parâmetro em vez de processar dados lidos do teclado (entrada padrão). Por exemplo, ao invocar `sscanf("fragata 2", "%s %d", tipo, &x)`, a variável `tipo` (vetor de caracteres) irá armazenar a string “fragata”, enquanto que a variável `x` (inteiro) irá armazenar o inteiro 2.
2. Considere que os strings no arranjo `navios` estão terminados com o caractere nulo `\0`.
3. Utilize a função `int strcmp(const char * str1, const char * str2)` para comparar dois strings. A função `strcmp` retorna zero se, e somente se, os strings `str1` e `str2` são iguais.
4. Considere que os navios a serem posicionados no tabuleiro estão todos em posições válidas (isto é, não ultrapassam as bordas do tabuleiro). Você não precisa fazer verificação de erros, basta posicionar os navios no tabuleiro.

```
void posicionar_navios(char tabuleiro[12][12], char* navios[], int n) {
```

```
}
```

Questão 2 – Verificar Tiro (4 pontos)

Implemente uma função em C chamada `verificar_tiro` que recebe como parâmetros:

- Uma matriz `tabuleiro` de 12x12 do tipo `char`, representando o tabuleiro do jogo.
- Dois inteiros `linha` e `coluna`, representando a coordenada do tiro do jogador.

Sua função deve verificar se o tiro acertou um navio. Se acertou, deve **atualizar o tabuleiro** para refletir o novo estado do jogo e retornar 1 (acertou). Caso contrário, deve retornar zero (errou).

Questão 3 – Verificar Término do Jogo (4 pontos)

Implemente uma função em C chamada `jogo_acabou` que recebe como parâmetro o tabuleiro 12x12 e verifica se o jogo acabou. Como definido anteriormente, o jogo acaba quando todos os navios estiverem afundados. Se o jogo acabou, retorne 1. Caso contrário, retorne zero.

Questão 4 – Função Principal do Jogo (4 pontos)

Complete as lacunas no código a seguir **utilizando as funções desenvolvidas nas questões anteriores** para implementar a lógica de controle do jogo. Para facilitar o processo de entrada dos dados, vamos considerar que as especificações de navios foram recebidas como parâmetros na linha de comando (`argc` e `argv`). A especificação de um navio em um `argv[x]` (para qualquer $x > 1$) contém um string com três campos, conforme descrito na Questão 1 e processado pela função `posicionar_navios`. Exemplo de entrada: `jogo.exe "fragata 2 3" "corveta 4 5" "porta-aviao 6 7"`. Para essa entrada, `argc` é 4 e `argv[1]` é `"fragata 2 3"`. Considere também que a função `verifica_navios` já está implementada e verifica quaisquer erros nos parâmetros recebidos. Finalmente, as posições dos tiros devem ser lidas do teclado a cada rodada e armazenadas nas variáveis `linha` e `coluna` já declaradas.

```
int main(void) {
    char tabuleiro[12][12];
    if(!verifica_navios(argv, argc)) {
        printf("Erro na especificação dos navios!\n");
        return 1;
    }
    // passando navios a partir do argv[1], pois o argv[0]
    // contém o nome do executavel:
    posicionar_navios(tabuleiro, &argv[1], argc-1);
    while (1) {
        int linha, coluna;
        printf("Digite as coordenadas do tiro (linha coluna): ");

        -----;

        if (-----) {
            printf("Acertou!\n");
        } else {
            printf("Errou!\n");
        }

        if (-----) {
            printf("Voce afundou todos os navios!\n");
            printf("Fim de jogo.\n");

            -----;
        }

    }
    return 0;
}
```

Parte 2 – Triângulo de Pascal (10 pontos)

O Triângulo de Pascal é formado por coeficientes binomiais e seus elementos podem ser calculados facilmente quando organizados em um triângulo. Veja um Triângulo de Pascal com 6 linhas:

1	Note que em cada linha (exceto a primeira):
1 1	1. O primeiro e o último elemento são sempre iguais a 1
1 2 1	2. Os demais elementos são dados pela soma de dois elementos na linha anterior. Mais precisamente, o i -ésimo elemento na linha L é dado pela soma do i -ésimo elemento da linha $(L-1)$ e seu antecessor. Por exemplo,
1 3 3 1	o primeiro elemento 10 na última linha ($i = 3$) é o resultado da soma de 6
1 4 6 4 1	($i = 3$) e 4 ($i = 2$) na linha anterior. O segundo elemento 10 na última linha
1 5 10 10 5 1	($i = 4$) é o resultado da soma de 4 ($i = 4$) e 6 ($i = 3$) na linha anterior.

Escreva uma **função** que recebe como parâmetro um inteiro N e imprime na tela um Triângulo de Pascal de N linhas. Considere que o valor máximo de N é 1024.